

```
import pandas as pd
df=pd.read_csv("medical_insurance.csv")
df.columns.tolist()
print(df.head(7))
print(df.shape)
print(df.info())
print(df.describe())
```

```
   age  sex  bmi  children  smoker  region  charges
0   19  female  27.900      0    yes  southwest  16884.92400
1   18   male  33.770      1    no   southeast  1725.55230
2   28   male  33.000      3    no   southeast  4449.46200
3   33   male  22.705      0    no  northwest  21984.47061
4   32   male  28.880      0    no  northwest  3866.85520
5   31  female  25.740      0    no   southeast  3756.62160
6   46  female  33.440      1    no   southeast  8240.58960
```

```
(2772, 7)
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 2772 entries, 0 to 2771
```

```
Data columns (total 7 columns):
```

```
#   Column  Non-Null Count  Dtype
---  ---
0   age      2772 non-null    int64
1   sex       2772 non-null    object
2   bmi       2772 non-null    float64
3   children  2772 non-null    int64
4   smoker    2772 non-null    object
5   region    2772 non-null    object
6   charges   2772 non-null    float64
```

```
dtypes: float64(2), int64(2), object(3)
```

```
memory usage: 151.7+ KB
```

```
None
```

```
      age      bmi  children  charges
count  2772.000000  2772.000000  2772.000000  2772.000000
mean    39.109668    30.701349    1.101732   13261.369959
std     14.081459     6.129449    1.214806   12151.768945
min     18.000000    15.960000     0.000000    1121.873900
25%     26.000000    26.220000     0.000000    4687.797000
50%     39.000000    30.447500     1.000000    9333.014350
75%     51.000000    34.770000     2.000000   16577.779500
max     64.000000    53.130000     5.000000   63770.428010
```

```
numerical_cols=df.select_dtypes(include=['int64','float64']).columns
print("NUMERICAL DATA")
print(list(numerical_cols))
print("\n")
categorical_cols=df.select_dtypes(include=['object']).columns
print("CATEGORICAL DATA")
print(list(categorical_cols))
```

```
NUMERICAL DATA
['age', 'bmi', 'children', 'charges']
```

```
CATEGORICAL DATA
['sex', 'smoker', 'region']
```

```
print(df.isnull().sum())
print(df.duplicated().sum())
```

```
age      0
sex      0
bmi      0
children 0
smoker   0
region   0
charges  0
dtype: int64
1435
```

```
df.drop_duplicates(inplace=True)
print(df.duplicated().sum())
```

```
0
```

Why Normalization is Required for the Selected Dataset

The Medical Insurance Cost Prediction dataset contains numerical attributes such as age, bmi, children, and charges. These attributes have different ranges of values. For example, age ranges from 18 to 64, while charges can range from a few thousand to several tens of thousands.

If the data is used without normalization, attributes with larger values may dominate those with smaller values, affecting the performance of machine learning algorithms. Normalization brings all numerical attributes to a common scale, ensuring that each feature contributes equally to the analysis.

Normalization also improves model accuracy, speeds up the training process, and helps machine learning algorithms perform more effectively. Therefore, normalization is an important preprocessing step for the Medical Insurance Cost Prediction dataset.

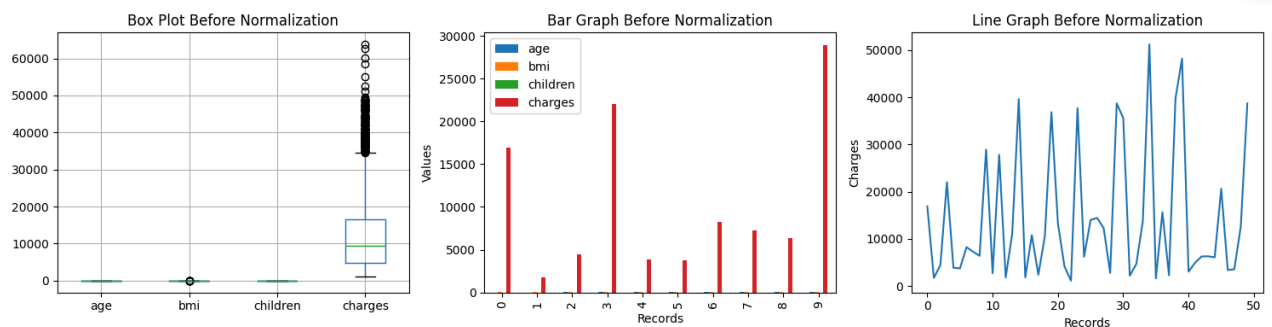
```
import matplotlib.pyplot as plt

plt.figure(figsize=(15,4))

# Box Plot
plt.subplot(1,3,1)
df[df.columns].boxplot()
plt.title("Box Plot Before Normalization")

# Bar Graph
plt.subplot(1,3,2)
df[df.columns].head(10).plot(kind='bar', ax=plt.gca())
plt.title("Bar Graph Before Normalization")
plt.xlabel("Records")
plt.ylabel("Values")

# Line Graph
plt.subplot(1,3,3)
plt.plot(df['charges'][:50])
plt.title("Line Graph Before Normalization")
plt.xlabel("Records")
plt.ylabel("Charges")
plt.tight_layout()
plt.show()
```



```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

minmax_df = pd.DataFrame(
    scaler.fit_transform(df[['age', 'bmi', 'children', 'charges']]),
    columns=['age', 'bmi', 'children', 'charges']
)
```

```
import matplotlib.pyplot as plt

plt.figure(figsize=(15,4))

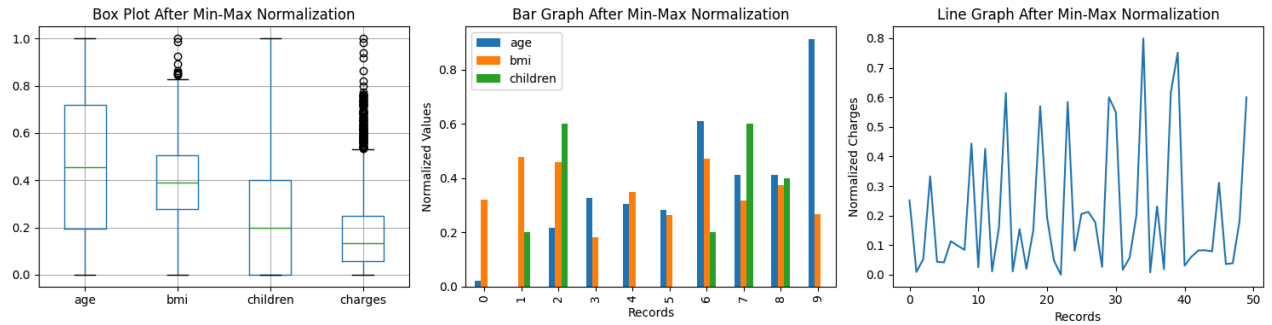
# Box Plot
plt.subplot(1,3,1)
minmax_df.boxplot()
plt.title("Box Plot After Min-Max Normalization")

# Bar Graph
plt.subplot(1,3,2)
minmax_df[['age', 'bmi', 'children']].head(10).plot(kind='bar', ax=plt.gca())
plt.title("Bar Graph After Min-Max Normalization")
plt.xlabel("Records")
plt.ylabel("Normalized Values")

# Line Graph
plt.subplot(1,3,3)
```

```
plt.plot(minmax_df['charges'][:50])
plt.title("Line Graph After Min-Max Normalization")
plt.xlabel("Records")
plt.ylabel("Normalized Charges")

plt.tight_layout()
plt.show()
```



```
from sklearn.preprocessing import MinMaxScaler, StandardScaler
import pandas as pd
import matplotlib.pyplot as plt

# Numerical attributes
num_cols = ['age', 'bmi', 'children', 'charges']

# =====
# MIN-MAX NORMALIZATION
# =====

minmax_scaler = MinMaxScaler()

minmax_df = pd.DataFrame(
    minmax_scaler.fit_transform(df[num_cols]),
    columns=num_cols
)

print("MIN-MAX NORMALIZATION")
print(minmax_df.head())

print("\nORIGINAL VS MIN-MAX NORMALIZED VALUES")
comparison_minmax = pd.concat(
    [df[num_cols].head(), minmax_df.head()],
    axis=1,
    keys=['Original', 'Min-Max']
)
print(comparison_minmax)

# Visualization After Min-Max
plt.figure(figsize=(15,4))

plt.subplot(1,3,1)
minmax_df.boxplot()
plt.title("Box Plot After Min-Max")

plt.subplot(1,3,2)
minmax_df[['age', 'bmi', 'children']].head(10).plot(
    kind='bar',
    ax=plt.gca()
)
plt.title("Bar Graph After Min-Max")

plt.subplot(1,3,3)
plt.plot(minmax_df['charges'][:50])
plt.title("Line Graph After Min-Max")

plt.tight_layout()
plt.show()

# =====
# Z-SCORE NORMALIZATION
# =====
```

```

zscore_scaler = StandardScaler()

zscore_df = pd.DataFrame(
    zscore_scaler.fit_transform(df[num_cols]),
    columns=num_cols
)

print("\n\nZ-SCORE NORMALIZATION")
print(zscore_df.head())

print("\nMEAN AFTER Z-SCORE")
print(zscore_df.mean())

print("\nSTANDARD DEVIATION AFTER Z-SCORE")
print(zscore_df.std())

# Distribution Analysis

```

MIN-MAX NORMALIZATION

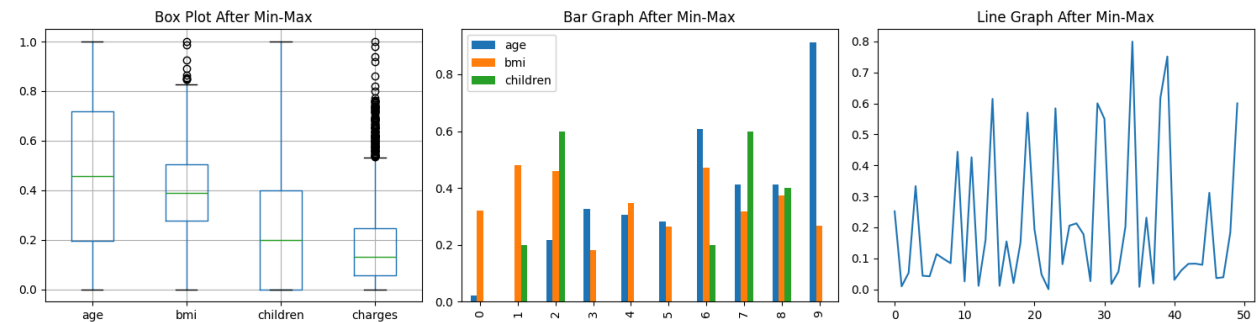
	age	bmi	children	charges
0	0.021739	0.321227	0.0	0.251611
1	0.000000	0.479150	0.2	0.009636
2	0.217391	0.458434	0.6	0.053115
3	0.326087	0.181464	0.0	0.333010
4	0.304348	0.347592	0.0	0.043816

ORIGINAL VS MIN-MAX NORMALIZED VALUES

Original					Min-Max				
	age	bmi	children	charges		age	bmi	children	
0	19	27.900	0	16884.92400	0.021739	0.321227	0.0		
1	18	33.770	1	1725.55230	0.000000	0.479150	0.2		
2	28	33.000	3	4449.46200	0.217391	0.458434	0.6		
3	33	22.705	0	21984.47061	0.326087	0.181464	0.0		
4	32	28.880	0	3866.85520	0.304348	0.347592	0.0		

charges

0	0.251611
1	0.009636
2	0.053115
3	0.333010
4	0.043816



Z-SCORE NORMALIZATION

	age	bmi	children	charges
0	-1.440418	-0.453160	-0.909234	0.297857
1	-1.511647	0.509422	-0.079442	-0.954381
2	-0.799350	0.383155	1.580143	-0.729373
3	-0.443201	-1.305052	-0.909234	0.719104
4	-0.514431	-0.292456	-0.909234	-0.777499

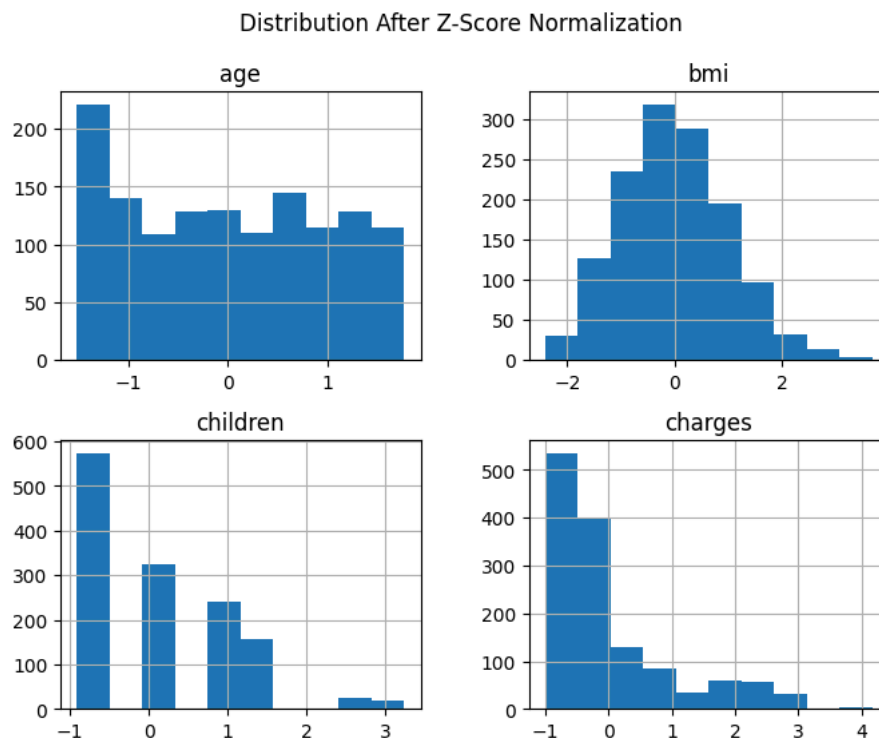
MEAN AFTER Z-SCORE

age	-6.643070e-17
bmi	-2.285216e-16
children	3.852980e-17
charges	-1.009747e-16
dtype:	float64

STANDARD DEVIATION AFTER Z-SCORE

age	1.000374
bmi	1.000374
children	1.000374
charges	1.000374
dtype:	float64

```
zscore_df.hist(figsize=(8,6))
plt.suptitle("Distribution After Z-Score Normalization")
plt.show()
```



Comparison of Normalization Techniques

Min-Max Normalization: • Scales numerical values between 0 and 1. • Preserves the original relationships among data points. • Easy to interpret and suitable for algorithms that require bounded values. • Sensitive to outliers because extreme values affect the scaling range.

Z-Score Normalization: • Transforms data to have a mean of 0 and a standard deviation of 1. • Standardizes features with different scales. • Less sensitive to outliers compared to Min-Max Normalization. • Suitable for statistical analysis and machine learning algorithms.

Comparison: • Min-Max Normalization produces values in a fixed range (0 to 1). • Z-Score Normalization produces values centered around zero with no fixed range. • Min-Max is useful when the data has no significant outliers. • Z-Score is preferred when the dataset contains variations and extreme values.

Most Suitable Method for the Healthcare Domain

For the Medical Insurance Cost Prediction dataset, Z-Score Normalization is the most suitable method.

Justification: • The charges attribute contains large variations and possible outliers. • Healthcare datasets often have features with different scales and distributions. • Z-Score Normalization standardizes the data using mean and standard deviation. • It reduces the effect of extreme values and improves the performance of machine learning models.

Conclusion

Both normalization techniques successfully transformed the numerical attributes. Min-Max Normalization scaled the values between 0 and 1, whereas Z-Score Normalization standardized the data to have a mean of 0 and a standard deviation of 1. For the Medical Insurance Cost Prediction dataset, Z-Score Normalization is more suitable because it handles variations and outliers more effectively.